Evaluating Coding Agents for Robot Manipulation with a Sandboxed High-Level Interface

Lucas F. R. Mazzetto
University of São Paulo
São Carlos, Brazil
lucas.mazzetto@usp.br

Matheus S. Soares
University of São Paulo
São Carlos, Brazil
matheus.sousa.soares@usp.br

Ricardo T. Xavier
University of São Paulo
São Carlos, Brazil
taoni@usp.br

Glauco A. P. Caurin
University of São Paulo
São Carlos, Brazil
gcaurin@sc.usp.br

***Abstract*—Recent advances in large language models make it practical to generate executable robot programs from natural-language instructions. This paper investigates whether coding agents can perform manipulation through a bounded high-level interface. We evaluate Claude Code and Codex in a simulated environment in which perception, motion planning, and control are handled by the robot-side execution system, while the agent operates in an isolated sandbox. We conduct experiments to measure how persistent skills affect task completion time and success rate, how reliably agents can grasp and manipulate objects, and whether they can accurately judge task completion. With persistent skills, Claude Code achieved an average task success rate of 81% and Codex 63%, compared with 48% and 37% without them. On basic pick-and-place tasks, reusing learned skills reduced Claude Code's mean trial time from 14–29 minutes to 3–6 minutes. Despite these improvements, grasping some objects and moving every object in a scene remained challenging. At the same time, agents could not always tell whether a task was completed correctly, making independent evaluation feedback necessary to show them whether their actions were right or wrong and guide future attempts.**

***Index Terms*—Code-as-Policy, Robot Programming, Large Language Models, Coding Agents, Robot Manipulation**

I. Introduction

Robot actions have long been implemented as software manually written by human developers. Recent advances in large language models (LLMs), however, have made it increasingly practical to synthesize such programs from natural-language instructions and observations of the environment. In the Code-as-Policy paradigm, a language or vision-language model (VLM) generates an executable program that encodes how a robot interprets available information, selects and organizes actions, and produces behavior. This program provides an interpretable and reusable representation of the robot's policy and, in some systems, can use execution feedback to verify or adapt subsequent actions [1]–[3].

At the same time, autonomous code generation exposes a practical tension. Robot software frameworks offer rich perception, planning, and control capabilities. However, unrestricted access to these capabilities can lead to generated programs that are unsafe or dependent on low-level implementation details. Recent work suggests that robot-oriented language models are more effective when they operate through clear, high-level robot functions, while benchmarks show that manipulation performance degrades when agents must directly compose low-level perception, geometry, and control interfaces [4], [5]. Therefore, practical robot coding agents need a constrained harness that provides a limited set of well-defined robot actions, gives clear feedback about their execution, and restricts generated code to the approved interface [6].

In such a harness, reusable code and skills become the primary mechanism for expressing useful agent behavior across tasks. They provide routines for perception, action, recovery, and task decomposition that can be acquired, reused, and adapted instead of regenerated on every request [7], [8]. The same boundary that enables skill reuse also supports safe execution. Since skills and generated scripts are explicit text and programs over a limited interface, they can be checked, simulated, tested against expected feedback, and verified against constraints before deployment. Generated code therefore becomes an auditable artifact, while the environment enforces safety rules even when the program cannot be guaranteed correct in advance [4], [9].

In this context, this paper investigates whether coding agents can generate executable programs that control a simulated robot to perform manipulation tasks through such a bounded interface. In particular, the proposed harness isolates the programming agent from the robot's runtime by ensuring that the runtime takes care of perception, planning, control, and execution. The sandbox exposes only a higher-level interface for observing the scene, sending predefined commands, and receiving clear execution feedback. In this environment, the evaluation is designed to answer three questions: Can off-the-shelf coding agents complete manipulation tasks through the bounded interface? Do persistent skills help agents improve over repeated attempts, and can they reuse those skills on new, more difficult tasks? Can agents accurately judge whether they completed a task?

II. Background and Related Work

Code-as-Policy methods use language or vision-language models to translate natural-language instructions into executable robot programs, which call predefined perception and control application programming interfaces (APIs) and combine them with standard programming constructs such as

This work was supported by Petróleo Brasileiro S/A - Petrobras, using resources from the ANP R&D clause, in partnership with the University of São Paulo (USP) and the Fundação de Apoio à Física e à Química (FAFQ), under Cooperation Agreements #2023/00016-6 and #2023/00013-7.

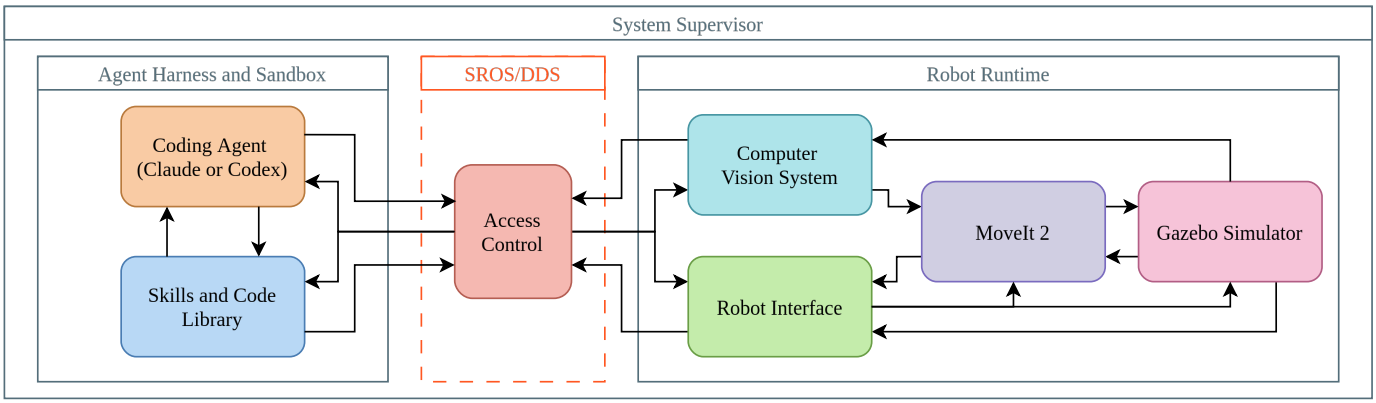


Fig. 1. Secure code-generation harness. The coding agent and skill library run in an isolated sandbox and access the robot runtime through Secure ROS 2 (SROS2) and the Data Distribution Service (DDS). The system supervisor orchestrates trials and independently verifies outcomes.

loops and conditionals [1]. That way, the model can still use reasoning to choose objects and order actions when given the API specification, the objects in the scene, and a few examples, but each generated step is still limited to the robot's API [2]. This means that rather than being merely a string of language tokens or motor commands, the robot's behavior is encoded as a program that acts on the robot exclusively through the secured API and can be analyzed, tested, and reused.

However, a program being valid and executable does not mean it is correct or safe to run on a robot. Some architectures therefore provide only high-level functions and execution feedback, without direct access to the actuators, since such interfaces make actions easier to verify and shape what the agent can do [4], [6]. The interface is also important because LLM or VLM agents may hallucinate calls, misunderstand instructions, or trigger unintended actions. In robotics, such errors can cause not only data loss and security breaches, but also result in unintended physical motion, where the same failure can end in an accident. Code generated for a robot cannot, therefore, be presumed to be safe merely because it is valid and was created using an advanced language model [10].

Two complementary techniques commonly used to address these issues are sandboxing and runtime enforcement. Sandboxing allows testing of the agent in isolation, for example, using a simulator before it is deployed. This makes it possible to identify unexpected or unsafe behavior without putting the robot or nearby people at risk [10]. Runtime enforcement provides an additional layer of protection during execution by checking each action, for example, through monitoring or auditing, before permitting it to proceed. For robots, these two techniques combine into an execution harness, a controlled environment that runs the generated program, limits the robot functions it may call, and helps prevent unsafe actions during operation [11], [12].

Within the harness, the agent can go beyond calling the predefined functions one by one and compose them into skills, which are reusable programs and instructions, written by the agent itself, that implement higher-level routines such as finding an object, choosing a grasp, or recovering from a failure [4]. Each skill is stored as a file that combines both code and instructions in natural language on when and how to use it, so it can be inspected, corrected, and versioned like any other software artifact, while still acting only through the harness interface and its bounds. Stored skills serve as the agent's memory, since a growing library of solved routines removes the need to reason from scratch on every task [5], and agents can reuse such a library on new tasks and refine their programs using environment feedback [13].

Drawing on these ideas, this work evaluates coding agents that operate a simulated robot arm through a secure and enforced interface while progressively building and reusing a skill library over tasks of increasing difficulty.

III. Methodology

For reliable testing of coding agents for robot manipulation, it is necessary to have a controlled robot system, a bounded interface, and an independent outcome verification. In this work, we focused on whether agents could use high-level robot interfaces to execute manipulation tasks and develop reusable skills across attempts. We combined a simulated environment, an agent sandbox, persistent skill conditions, and a ground-truth evaluator to measure task performance and determine whether the agents' own success reports matched the actual outcomes.

A. System Harness and Sandbox

There are three main components of the system: a simulation environment, an agent sandbox, and an experiment supervisor, as shown in Fig. 1. The agent generates high-level policies, while perception, motion planning, and control stay in the simulation environment. The visible interface to the agent, base prompt, and time limit are fixed across runs.

The simulation uses Gazebo Harmonic with DART [14] as the physics engine, Robot Operating System 2 (ROS 2), and MoveIt 2 with a simulated Franka Emika Panda arm (Fig. 2). We used OMPL's RRTConnect [15] for collision-free path planning, KDL [16] for inverse kinematics and

`ros2_control` [17] for trajectory execution. Grasped objects are not rigidly attached in simulation. They are held only by the friction generated at the contact between the gripper fingers and the object. Three stereo cameras with red-green-blue-depth (RGB-D) sensing provide RGB images, depth data, and registered point clouds from different views of the table. The left sensor of each stereo camera runs a You Only Look Once (YOLO) instance-segmentation model pretrained on the Common Objects in Context (COCO) dataset [18], [19], producing object masks and detections. Depth information projects these detections into 3D bounding boxes in the world frame. The vision system then merges detections of the same object across cameras and fuses the point clouds into a common scene representation. The fused detections are provided to the agent so it can identify and select objects. At the same time, the motion planner uses the scene geometry to plan collision-aware robot motions, while the agent can access only the high-level interface and not the planner itself.

Through this interface, the agent can issue high-level commands to move the arm, operate the gripper, and transfer detected objects between specified locations while receiving execution feedback. The simulation environment then converts these commands into planned and controlled motions, and a simulated finger-effort proxy verifies whether a grasp was achieved. To maintain this separation between the agent and the robot runtime, the coding agent runs in a separate environment protected by Secure ROS 2 (SROS2) and Data Distribution Service (DDS) security. Within this environment, it can receive selected camera, pose, gripper, and execution-feedback topics and call the permitted manipulation services and actions. However, it cannot access joint states, low-level controllers, or exact object poses. The agent's reusable skills are stored as files containing instructions and scripts in this separate environment.

Before each trial, the system supervisor generated the scene, restarted the simulation, and waited until the system became healthy. Once the system was ready, a fresh agent session received the task and made one attempt without access to the final evaluation. After the attempt, the supervisor measured the trial duration, the positions of all objects, and the agent's self-reported outcome. A trial is considered successful only if the requested manipulation goal is achieved, none of the undesired objects is moved into the basket, and the basket itself does not move outside the specified displacement area (within 5 cm of its initial position). Accordingly, success rates and outcome classes are based on the supervisor rather than on the agent's claim. Agent judgment is evaluated by comparing its self-report with the supervisor verdict.

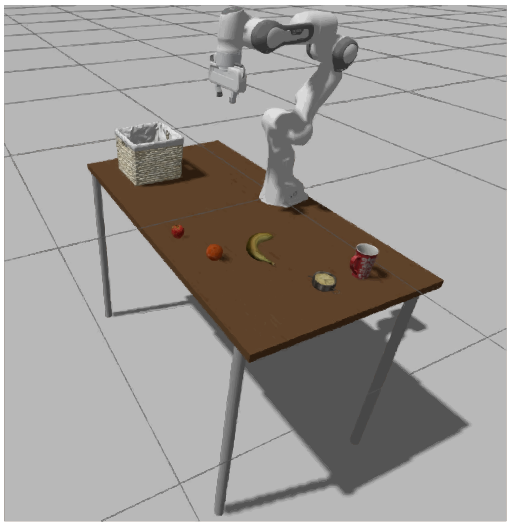


Fig. 2. Simulated tabletop manipulation scene with the Franka Panda arm, fruit and non-fruit task objects, and a basket serving as the placement target.

B. Experiments

Using the sandboxed harness described in the System Harness and Sandbox subsection (Section III-A), we evaluated Claude Code, running Claude Opus 5.0, and Codex, using GPT-5.6 Sol, as robot-policy authors. The simulator, robot platform, base prompt, client-side interface, middleware authorization, and medium thinking-effort setting were identical for both agents. Under these fixed conditions, the evaluation examined baseline capability, skill reuse, and whether skills learned in one task helped in later tasks. Because each model was used through its own coding agent tool, the comparison is between complete agent systems rather than between the underlying language models.

We organized the evaluation into three task groups with increasing difficulty. The first group tested basic pick-and-place actions, the second required the agent to select objects based on their properties or positions, and the third tested whether the agent could perform several manipulations while maintaining the correct state of the scene. Each trial provided a natural-language instruction and a different arrangement of objects on the table. The task groups and their individual tasks are described in more detail below.

1) **Basic pick and place:** These tasks provided a baseline for object manipulation with minimal reasoning. Each scene contained only the target object and the basket, with no distractor objects present. The agent had to identify the named object and execute a pick-and-place action using a feasible grasp pose. The group included:
   a) Pick the orange and place it in the basket.
   b) Pick the apple and place it in the basket.
   c) Pick the banana and place it in the basket.
2) **Attribute and spatial relations:** The aim of these tasks was to evaluate whether the agent could identify an item based on its properties or position rather than on class name alone. The generated code had to compare detections by size or position, and success required placing the object that satisfied the instruction in the basket. The group contained:
   a) Among three oranges of different sizes, place the smallest one into the basket.
   b) Put the object closest to the basket into the basket.

c) Pick the leftmost object on the table and place it in the basket. The leftmost object alternates between two classes across trials, so the rule cannot be reduced to a class name.

3) **Multi-step manipulation:** These tasks evaluated whether the agent could compose multiple pick-and-place operations while maintaining the state of the scene and controlling the flow of actions. Success required placing all and only the requested objects in the basket. The group covered:
   a) Put all objects into the basket.
   b) Put all fruits into the basket and leave the other objects on the table.
   c) Put all non-fruit objects into the basket.

The tasks were also limited by the perceptual system. Since the object detector had been trained using the COCO dataset [18], [19], the movable objects were limited to recognizable classes. However, considering that the basket could not be detected as a normal COCO class, the agent had to detect the position of the basket from the camera view only.

To evaluate persistent skills, we compared two conditions: Cold and Reuse. In Cold, the agent had no access to skills from earlier attempts. In Reuse, skills persisted and were updated across trials and task groups. The conditions are defined below:

1) **Cold:** In each trial, the agent received an empty temporary skill library. It could not read skills from previous attempts, and the prompt did not ask it to record what it learned.
2) **Reuse:** The agent had access to a persistent skill library. Skills stored during one attempt remained available in later attempts and tasks. The library was initially empty for the first task group, carried into the next group, and updated throughout the experiment.

Runs were executed in task-group order, so skills accumulated under Reuse in one group were available in the next. Each task was scheduled for 12 Reuse iterations and 3 Cold trials run outside the Reuse sequence. This gives 108 Reuse and 27 Cold trials, totaling 135 per agent, or 270 scheduled trials across both agents.

C. Metrics

Each trial recorded the ground-truth outcome from the simulator, its duration, and the result claimed by the agent. For multi-object tasks, the outcome of each individual object was also recorded. The following measurements were made:

1) **Task success rate:** the percentage of trials classified as correct.
2) **Outcome class:** each trial was labeled as a success, incomplete target, distractor in the basket (an unrequested object that ended up in the basket), basket displacement, timeout, or system error.
3) **Trial time:** the total time taken for each attempt.
4) **Self-reported outcome:** the result claimed by the agent.
5) **Object placement:** in multi-step tasks, each object was marked correct if it ended up at its intended location, and incorrect otherwise.

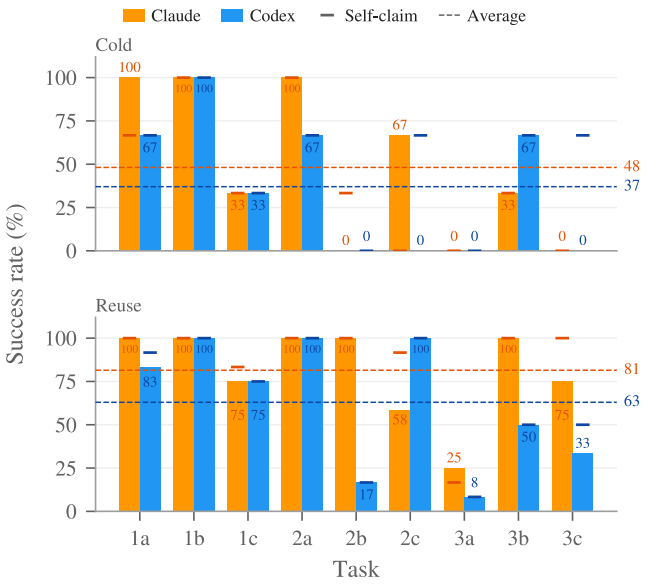


Fig. 3. Per-task success rates under Cold and Reuse. Bars show independently evaluated task success, tick marks show self-reported success, and dashed lines show each agent's average across tasks.

IV. Results

Overall, skill reuse improved task success for both agents, but some tasks remained difficult. Four Reuse trials were excluded for system issues: one Claude Code trial on task 1b and three Codex trials on tasks 1b, 2a, and 2c. Therefore, the analysis included 107 Claude Code and 105 Codex Reuse trials and all 54 Cold trials. Fig. 3 shows average task success rising from 48% to 81% for Claude Code and from 37% to 63% for Codex. However, moving every object remained the hardest task, with Reuse success rates of 25% and 8%, respectively. The agents' success claims also differed from actual outcomes: Claude Code claimed success in every Reuse trial of task 3c, although only 75% succeeded. To account for the dependence between successive Reuse trials, we computed 95% confidence intervals (CIs) with a task-level bootstrap that resamples whole tasks rather than individual trials. Reuse improved average success by 33 percentage points for Claude Code (95% CI [10, 58]) and by 26 for Codex ([8, 48]), whereas the difference between the agents was not significant (18 points, $[-4, 41]$).

Neither agent derived a grasp or contact model. Instead, both relied on trial-and-error heuristics encoded in their skills: approaching the center of the 3D bounding box, setting the gripper opening from the box width, estimating the grasp orientation from the principal axis of the object's point cloud, applying approach and lift offsets, and retrying after a failed grasp check.

The object-level outcomes in Fig. 4 provide a closer view of the multi-step tasks. Distractors remained outside the basket in the selective tasks, consistent with correct selection, but target placement remained uneven. In task 3a, for instance, Codex placed the banana correctly in only 8% of trials, while correct

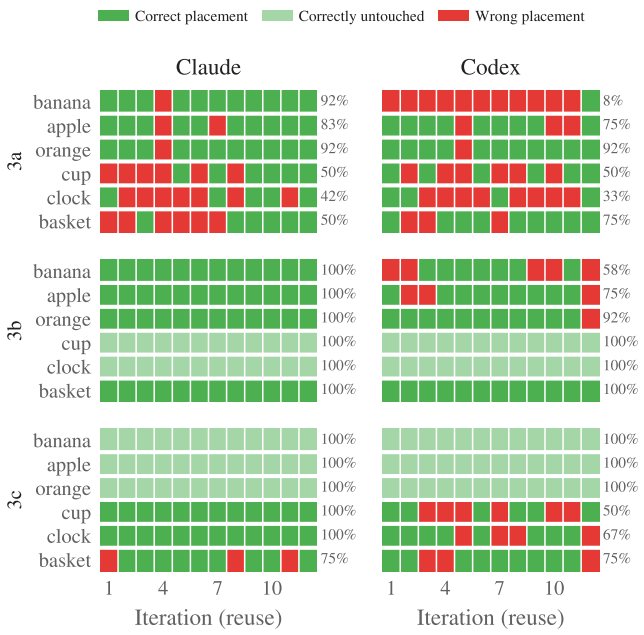


Fig. 4. Final object placement and basket stability across Reuse iterations of the multi-step tasks. Dark green indicates correct target placement, light green indicates distractors correctly left outside the basket, and red indicates an incorrect outcome.

cup and clock placement ranged from 33% to 50% across both agents. Even correct placement did not guarantee task success. Claude Code placed both targets correctly throughout task 3c, yet basket displacement caused the failed trials.

Turning to trial duration, mean durations computed from the trials in Fig. 5 decreased from 14–29 to 3–6 minutes for Claude Code on basic pick-and-place tasks under Reuse, with smaller reductions for Codex. Both agents also succeeded on task 2a from their first Reuse trial, suggesting skill transfer to related tasks. However, shorter attempts were not always successful, and moving every object remained unreliable. On task 3a, Claude Code first succeeded at iteration 9 and Codex at iteration 12, showing that improvement was limited and inconsistent.

V. Conclusion

This paper examined whether coding agents can control a robot through a sandboxed programming interface and whether a persistent skill library helps them improve across repeated and related tasks. Both agents completed a range of pick-and-place and spatial tasks, and reusing learned skills significantly improved average task success and often made attempts faster. However, the difference between the agents was not statistically significant, the benefits varied by task, and easier tasks were not always successful. Skills also appeared to transfer to related tasks, but moving every object remained unreliable for both agents. In selective tasks, non-target objects remained outside the basket, suggesting correct selection. Nevertheless, some targets remained difficult to pick and place, and even complete placement could result in failure if the basket moved. Despite the gains from skill reuse, these remaining difficulties highlight the need for more reliable manipulation capabilities.

The agents also could not reliably judge whether they had completed a task. They sometimes claimed success after failed trials, so their success claims cannot be treated as proof of completion. This creates a risk for skill learning, because an agent may save a skill after mistakenly judging an unsuccessful attempt as successful. Independent evaluation and clear feedback are therefore important safeguards for verifying task outcomes and guiding skill updates.

Future work should combine high-level code generation with low-level grasping capabilities, such as models that predict suitable grasp poses for individual objects. The system should also provide direct, independent feedback on task completion, including object placement and basket stability. With this information, agents could correct unsuccessful behavior and update their skills using verified outcomes, potentially improving manipulation accuracy while preserving the auditability of generated programs.

References

[1] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 9493–9500.

[2] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, "ProgPrompt: Generating situated robot task plans using large language models," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 11 523–11 530.

[3] Y. Mu, J. Chen, Q.-L. Zhang, S. Chen, Q. Yu, C. Ge, R. Chen, Z. Liang, M. Hu, C. Tao, P. Sun, H. Yu, C. Yang, W. Shao, W. Wang, J. Dai, Y. Qiao, M. Ding, and P. Luo, "RoboCodeX: Multimodal code generation for robotic behavior synthesis," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, Eds., vol. 235. PMLR, 21–27 Jul 2024, pp. 36 434–36 454. [Online]. Available: https://proceedings.mlr.press/v235/mu24a.html

[4] S. H. Vemprala, R. Bonatti, A. Bucker, and A. Kapoor, "ChatGPT for robotics: Design principles and model abilities," *IEEE Access*, vol. 12, pp. 55 682–55 696, 2024.

[5] L. Fu, J. Yu, K. El-Refai, E. Kou, H. Xue, H. Huang, W. Xiao, L. Fei-Fei, G. Shi, J. Wu, S. S. Sastry, Y. Zhu, K. Goldberg, and L. Fan, "CaP-X: A framework for benchmarking and improving coding agents for robot manipulation," in *Forty-third International Conference on Machine Learning*, 2026. [Online]. Available: https://openreview.net/forum?id=4JRO9plGAI

[6] J. Yang, C. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," *Advances in Neural Information Processing Systems*, vol. 37, pp. 50 528–50 652, 2024.

[7] H. Chen, S. Liu, Z. Li, D. Navarro-Alarcon, and P. Zheng, "Multimodal autonomous robotic long-horizon task planning via embodied language model and behavior trees," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 20 283–20 290.

[8] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, S. Yau, Z. Lin, L. Zhou *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 23 247–23 275.

[9] Y. Jin, D. Li, Y. A, J. Shi, P. Hao, F. Sun, J. Zhang, and B. Fang, "RobotGPT: Robot manipulation learning from ChatGPT," *IEEE Robotics and Automation Letters*, vol. 9, no. 3, pp. 2543–2550, 2024.

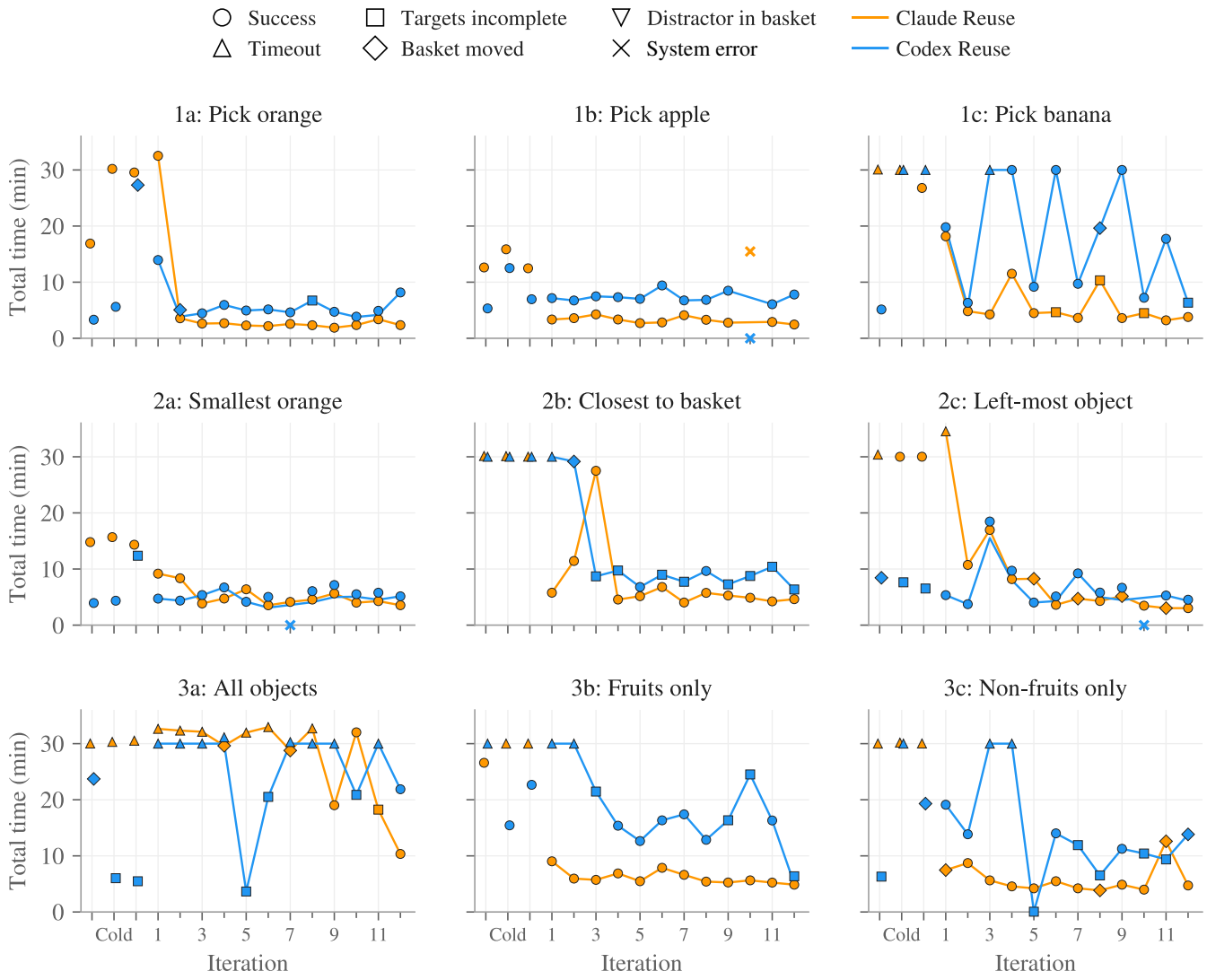


Fig. 5. Trial duration and outcomes across Cold trials and successive Reuse iterations. Orange represents Claude Code and blue represents Codex. Marker shapes indicate task outcomes, and solid lines connect recorded Reuse durations.

[10] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. Maddison, and T. Hashimoto, "Identifying the risks of LM agents with an LM-emulated sandbox," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 27 031–27 098.

[11] R. Greenblatt, B. Shlegeris, K. Sachan, and F. Roger, "AI control: Improving safety despite intentional subversion," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, Eds., vol. 235. PMLR, 21–27 Jul 2024, pp. 16 295–16 336.

[12] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE)*. ACM, 2026, 12 pages. [Online]. Available: https://doi.org/10.1145/3744916.3764546

[13] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Transactions on Machine Learning Research*, 2024. [Online]. Available: https://openreview.net/forum?id=ehfRiF0R3a

[14] J. Lee, M. X. Grey, S. Ha, T. Kunz, S. Jain, Y. Ye, S. S. Srinivasa, M. Stilman, and C. K. Liu, "DART: Dynamic animation and robotics toolkit," *Journal of Open Source Software*, vol. 3, no. 22, p. 500, 2018.

[15] I. A. Şucan, M. Moll, and L. E. Kavraki, "The open motion planning library," *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, 2012.

[16] The Orocos Project, "Orocos Kinematics and Dynamics Library (KDL)," accessed: Oct. 12, 2026. [Online]. Available: https://www.orocos.org/kdl.html

[17] S. Chitta, E. Marder-Eppstein, W. Meeussen, V. Pradeep, A. Rodríguez Tsouroukdissian, J. Bohren, D. Coleman, B. Magyar, G. Raiola, M. Lüdtke, and E. Fernández Perdomo, "ros_control: A generic and simple control framework for ROS," *Journal of Open Source Software*, vol. 2, no. 20, p. 456, 2017.

[18] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *European conference on computer vision*. Springer, 2014, pp. 740–755.

[19] R. Varghese and M. Sambath, "YOLOv8: A novel object detection algorithm with enhanced performance and robustness," in *2024 International conference on advances in data engineering and intelligent computing systems (ADICS)*. IEEE, 2024, pp. 1–6.